

L28 - Array 2- One dimensional — Sorting and Searching

Department of Computer Science
University of Pretoria

21 May 2024

-**Search:** find a given item in an array, return the index of the item, or -1 if not found.

-**Sort:** rearrange the items in an array into some order (smallest to biggest, alphabetical order, etc.).

There are various methods (algorithms) for carrying out these common tasks.

Linear Search

- Very simple method.
- Compare first element to target value,if not found then compare second element to target value . . .
- Repeat until:
target value is found (return its index) or we run out of items (return -1).

Linear Search example

```
int searchList(int list[], int size, int value) {  
    int index = 0; // index to process the array  
    int position = -1; // position of target  
    bool found = false; // flag, true when target is found  
  
    while (index < size && !found) {  
        if (list[index] == value) { // found the target  
            found = true; // set the flag  
            position = index; // record which item  
        }  
        index++; // increment loop index  
    }  
    return position;  
}
```

Program that uses linear search

```
#include <iostream>
using namespace std;

// Function prototype
int searchList(int[], int, int);

// Function definition
int searchList(int list[], int size, int value) {
    for (int i = 0; i < size; i++) {
        if (list[i] == value) {
            return i;
        }
    }
    return -1;
}
```

Program that uses linear search continues

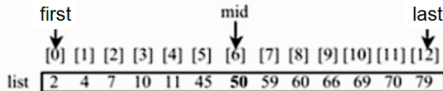
```
int main() {  
    const int SIZE = 5;  
    int idNums[SIZE] = {871, 750, 988, 100, 822};  
    int results, id;  
  
    cout << "Enter the employee ID to search for: ";  
    cin >> id;  
  
    results = searchList(idNums, SIZE, id);  
    if (results == -1) {  
        cout << "That ID number is not registered\n";  
    } else {  
        cout << "That ID number is found at location-" << results  
    }  
  
    return 0;  
}
```

Binary Search Algorithm

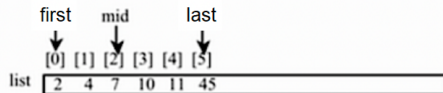
example

target is 11

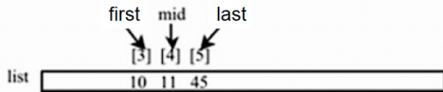
target < 50



target > 7



target == 11



Binary search in C++, what if first + last is odd or first == last?

```
int binarySearch(int array[], int size, int target) {  
    int first = 0; // index of the current first element  
    int last = size - 1; // index of the current last element  
    int middle; // index of the current middle element  
    int position = -1; // index of the target value  
    bool found = false; // flag  
  
    while (first <= last && !found) {  
        middle = (first + last) / 2; // calculate midpoint  
  
        if (array[middle] == target) {  
            found = true;  
            position = middle;  
        } else if (target < array[middle]) {  
            last = middle - 1; // search lower half  
        } else {  
            first = middle + 1; // search upper half  
        }  
    }  
    return position;  
}
```


Implement a function called `findStudentIndex` that accepts a student name as a parameter and returns the index of that student in the `studentNames` array. If the student is not found, the function should return -1. Use this function to search for the index of the student "Charlie" and display the result.

```
#include <iostream>
#include <string>
using namespace std;

// Function to find the index of a student in the studentNames
array
int findStudentIndex(string studentName, string studentNames[],
int numStudents) {
    for (int i = 0; i < numStudents; i++) {
        if (studentNames[i] == studentName) {
            return i;
        }
    }
    return -1; // Student not found
}

int main() {
    // Define an array of student names
    string studentNames[] = {"Alice", "Bob", "Charlie", "Diana", "Eve"};
    int numStudents = sizeof(studentNames) / sizeof(studentNames[0]);
```

```
// Define the student name to search for
string studentToFind = "Charlie";

// Call findStudentIndex
int index = findStudentIndex(studentToFind, studentNames,
                             numStudents);

// Print the result
if (index != -1) {
    cout << "Student-" << studentToFind
          << "-found-at-index-" << index
          << "." << endl;
}
else {
    cout << "Student-" << studentToFind << "-not-found." << endl;
}
return 0;
}
```

```
#include <iostream>
using namespace std;

int main() {
    const int SIZE = 5; // Size of the array
    int numbers[SIZE] = {10, 20, 5, 40, 30}; // Example array

    int count;
    int highest;

    // Assume the first element is the highest initially
    highest = numbers[0];
```

```
// Loop through the array starting from the second element
for (count = 1; count < SIZE; count++) {
    if (numbers[count] > highest) {
        highest = numbers[count];
    }
}

// Print the highest number found
cout << "The-highest-number-in-the-array-is:-" << highest << endl;

return 0;
}
```

Sorting Algorithms

- Sort: rearrange the items in an array into ascending or descending order.
- Selection Sort
- Bubble Sort



55 112 78 14 20 179 42 67 190 7 101 1 122 170 8 unsorted

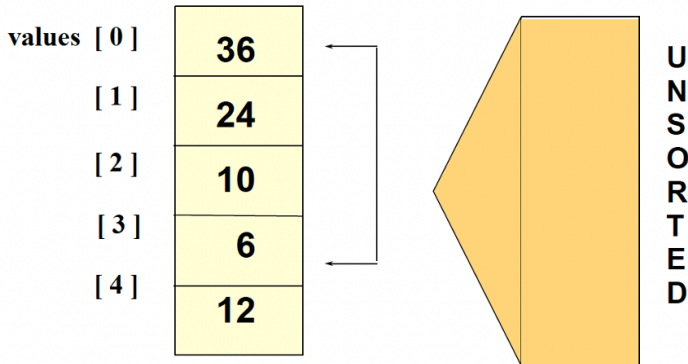
1 7 8 14 20 42 55 67 78 101 112 122 170 179 190 sorted

Why is sorting important?

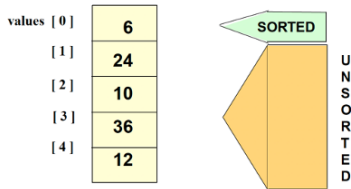
1. Searching in a sorted list is much easier than searching in an unsorted list.
2. Especially for people:
 - dictionary entries, phone book
 - card catalog in library (it used to be drawers of index cards)
 - bank statement: transactions in date order
3. Most data displayed by computers is sorted.

- There is a pass for each position ($0..size-1$)
- On each pass, the smallest (minimum) element in the rest of the list is exchanged (swapped) with element at the current position.
- The first part of the list (the part that is already processed) is always sorted
- Each pass increases the size of the sorted portion.

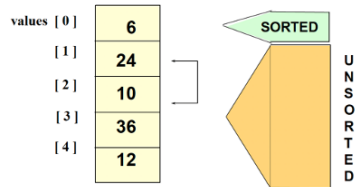
Selection Sort: Pass One



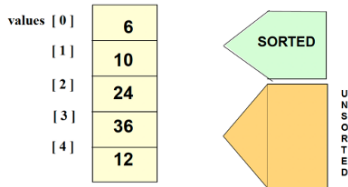
Selection Sort: End Pass One



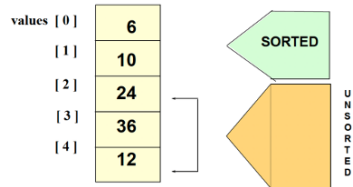
Selection Sort: Pass Two



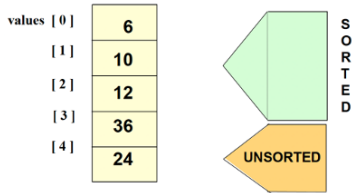
Selection Sort: End Pass Two



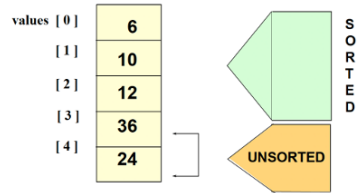
Selection Sort: Pass Three



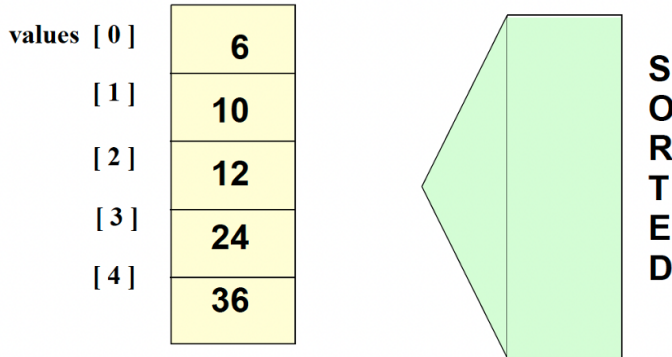
Selection Sort: End Pass Three



Selection Sort: Pass Four



Selection Sort: End Pass Four



```
// selection sort continues from previous page
#include <iostream>
using namespace std;

// Function for Selection sort
void selectionSort(int array[], int size)
{
    int currentIndex, nextIndex, minIndex;
    // One by one move boundary of
    // unsorted subarray
    for (currentIndex = 0; currentIndex < size - 1; currentIndex++)
        // Find the minimum element in
        // unsorted array
        minIndex = currentIndex;
        for (nextIndex = currentIndex + 1; nextIndex < size; nextIndex++)
            if (array[nextIndex] < array[minIndex])
                minIndex = nextIndex;
    }
    // Swap the found minimum element
    // with the first element
    if (minIndex != currentIndex)
        swap(array[minIndex], array[currentIndex]);
}
```

```
// Previous function continues
// Function to print an array
void printArray(int array[], int size)
{
    for (int i = 0; i < size; i++) {
        cout << array[i] << "-";
    }
    cout << endl;
}
// Driver program
int main()
{
    int array[] = { 64, 25, 12, 22, 11 };
    int size = sizeof(array) / sizeof(array[0]);
    // Function Call
    selectionSort(array, size);
    cout << "Sorted array:-\n";
    printArray(array, size);
    return 0;
}
```

```
//Another example:
#include <iostream>
using namespace std;
// Returns the index of the smallest element, starting at start
int findIndexOfMin(int array[], int size, int start) {
    int minIndex = start;
    for (int i = start + 1; i < size; i++) {
        if (array[i] < array[minIndex]) {
            minIndex = i;
        }
    }
    return minIndex;
}
// Sorts an array, using findIndexOfMin
void selectionSort(int array[], int size) {
    int temp;
    int minIndex;
    for (int index = 0; index < (size - 1); index++) {
        minIndex = findIndexOfMin(array, size, index);
        // Swap
        temp = array[minIndex];
        array[minIndex] = array[index];
        array[index] = temp;
    }
}
```



```
//Previous code continues
// Function to print an array
void printArray(int array[], int size) {
    for (int i = 0; i < size; i++) {
        cout << array[i] << "-";
    }
    cout << endl;
}
// Driver program
int main() {
    int array[] = { 64, 25, 12, 22, 11 };
    int size = sizeof(array) / sizeof(array[0]);
    // Function call
    cout << "Original-array:-";
    printArray(array, size);
    selectionSort(array, size);
    cout << "Sorted-array:-";
    printArray(array, size);

    return 0;
}
```

The Bubble Sort

- On each pass:
 - Compare first two elements. If the first is bigger, they exchange places (swap).
 - Compare second and third elements. If second is bigger, exchange them.
 - Repeat until last two elements of the list are compared.
- Repeat this process (keep doing passes) until a pass completes with no exchanges

Bubble sort

Example: first pass

- 7 2 3 8 9 1 $7 > 2$, swap
- 2 7 3 8 9 1 $7 > 3$, swap
- 2 3 7 8 9 1 $!(7 > 8)$, no swap
- 2 3 7 8 9 1 $!(8 > 9)$, no swap
- 2 3 7 8 9 1 $9 > 1$, swap
- 2 3 7 8 1 9 finished pass 1, did 3 swaps

Note: largest element is now in last position

Note: This is one complete pass!

Bubble sort

Example: second and third pass

- 2 3 7 **8** 1 9 2<3<7<8, no swap, !(8<1), swap
 - 2 3 7 1 8 9 (8<9) no swap
 - finished pass 2, did one swap
- 2 largest elements
in last 2 positions
-
- 2 3 **7** 1 8 9 2<3<7, no swap, !(7<1), swap
 - 2 3 1 7 8 9 7<8<9, no swap
 - finished pass 3, did one swap
- 3 largest elements
in last 3 positions

Bubble sort

Example: passes 4, 5, and 6

- 2 **3** 1 7 8 9 2<3, !(3<1) swap, 3<7<8<9
- 2 1 3 7 8 9
- finished pass 4, did one swap
- **2** 1 3 7 8 9 !(2<1) swap, 2<3<7<8<9
- 1 2 3 7 8 9
- finished pass 5, did one swap
- 1 2 3 7 8 9 1<2<3<7<8<9, no swaps
- finished pass 6, no swaps, list is sorted! ..

Bubble sort

how does it work?

- At the end of the first pass, the largest element is moved to the end (it's bigger than all its neighbors)
- At the end of the second pass, the second largest element is moved to just before the last element.
- The back end (tail) of the list remains sorted.
- Each pass increases the size of the sorted portion.
- No exchanges implies each element is smaller than its next neighbor (so the list is sorted).

Bubble Sort in C++

```
void bubbleSort (int array[], int size) {  
    bool swap;  
    int temp;  
  
    do {  
        swap = false;  
        for (int i = 0; i < (size-1); i++) {  
            if (array [i] > array[i+1]) {  
                temp = array[i];  
                array[i] = array[i+1];  
                array[i+1] = temp;  
                swap = true;  
            }  
        }  
    } while (swap);  
}
```

Program using bubble sort

```
#include <iostream>
using namespace std;

void bubbleSort(int [], int);
void showArray(int [], int);

int main() {
    int values[6] = {7, 2, 3, 8, 9, 1};

    cout << "The unsorted values are: \n";
    showArray (values, 6);

    bubbleSort (values, 6);

    cout << "The sorted values are: \n";
    showArray(values, 6);
}

void showArray (int array[], int size) {
    for (int i=0; i<size; i++)
        cout << array[i] << " ";
    cout << endl;
}
```

Output:

```
The unsorted values are:
7 2 3 8 9 1
The sorted values are:
1 2 3 7 8 9
```

In the following code take note of the following:

1. This loop runs through the unsorted part of the array. The size - 1 - pass condition ensures that we do not compare elements that are already sorted.
2. " pass " keeps track of how many elements are already sorted at the end of the array after each pass.


```
#include <iostream>
using namespace std;

// Function to sort the array in ascending order using bubble sort
void bubbleSort(int values[], int size) {
    int pass, index, temp;
    for (pass = 0; pass < size - 1; pass++) {
        for (index = 0; index < size - 1 - pass; index++) {
            if (values[index] > values[index + 1]) {
                // Swap the values
                temp = values[index];
                values[index] = values[index + 1];
                values[index + 1] = temp;
            } // end if
        } // end for index
    } // end for pass
}
```

```
int main() {  
    const int SIZE = 5; // Size of the array  
    int numbers[SIZE] = {10, 20, 5, 40, 30}; // Example array  
  
    // Call the bubbleSort function to sort the array  
    bubbleSort(numbers, SIZE);  
  
    // Print the sorted array  
    cout << "Array-sorted-from-lowest-to-highest:-";  
    for (int i = 0; i < SIZE; i++) {  
        cout << numbers[i] << "-";  
    }  
    cout << endl;  
    return 0;  
}
```

Sorting From highest to lowest

```
#include <iostream>
using namespace std;

// Function to sort the array in descending order
void sortDescending(int arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = i + 1; j < size; j++) {
            if (arr[i] < arr[j]) {
                // Swap the elements
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}
```

```
//Program continues from previous slide
int main() {
    const int SIZE = 5; // Initial size of the array
    int numbers[SIZE] = {10, 20, 5, 40, 30}; // Example array

    // Call the function to sort the array in descending order
    sortDescending(numbers, SIZE);

    // Print the sorted array
    cout << "Array-sorted-from-highest-to-lowest:-";
    for (int i = 0; i < SIZE; i++) {
        cout << numbers[i] << "-";
    }
    cout << endl;
    return 0;
}
```

Sorting From Lowest to Highest

```
#include <iostream>
using namespace std;

// Function to sort the array in ascending order using pointers
void sortAscending(int* arr, int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = i + 1; j < size; j++) {
            if (*(arr + i) > *(arr + j)) {
                // Swap the elements using pointers
                int temp = *(arr + i);
                *(arr + i) = *(arr + j);
                *(arr + j) = temp;
            }
        }
    }
}
```

```
//Program continues from previous slide
int main() {
    const int SIZE = 5; // Initial size of the array
    int numbers[SIZE] = {10, 20, 5, 40, 30}; // Example array

    // Call the function to sort the array in ascending order
    sortAscending(numbers, SIZE);
    // Print the sorted array
    cout << "Array-sorted-from-lowest-to-highest: -";
    for (int i = 0; i < SIZE; i++) {
        cout << numbers[i] << "-";
    }
    cout << endl;
    return 0;
}
```

Array Swapping using pointers

```
#include <iostream>
using namespace std;
```

```
// Function to sort the array in ascending order using pointers
void sortAscending(int* arr, int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = i + 1; j < size; j++) {
            if (*(arr + i) > *(arr + j)) {
                // Swap the elements using pointers
                int temp = *(arr + i);
                *(arr + i) = *(arr + j);
                *(arr + j) = temp;
            }
        }
    }
}
```

```
// Function to swap the values at two specified indices using pointers
void swapElements(int* arr, int index1, int index2) {
    int temp = *(arr + index1);
    *(arr + index1) = *(arr + index2);
    *(arr + index2) = temp;
}
```

```
// code continues from previous slide
int main() {
    const int SIZE = 5; // Initial size of the array
    int numbers[SIZE] = {10, 20, 5, 40, 30}; // Example array
    // Call the function to swap values 40 and 30
    int index1 = -1;
    int index2 = -1;
    // Find the indices of the values 40 and 30
    for (int i = 0; i < SIZE; i++) {
        if (numbers[i] == 40) {
            index1 = i;
        }
        if (numbers[i] == 30) {
            index2 = i;
        }
    }
    // Swap the elements if both values are found
    if (index1 != -1 && index2 != -1) {
        swapElements(numbers, index1, index2);
    }
}
```



```
// code continues from previous slide
// Print the array after swapping
cout << "Array-after-swapping-40-and-30:-";
for (int i = 0; i < SIZE; i++) {
    cout << numbers[i] << "-";
}
cout << endl;
// Call the function to sort the array in ascending order
sortAscending(numbers, SIZE);
// Print the sorted array
cout << "Array-sorted-from-lowest-to-highest:-";
for (int i = 0; i < SIZE; i++) {
    cout << numbers[i] << "-";
}
cout << endl;
return 0;
}
```

swapping without pointers

```
#include <iostream>
using namespace std;
// Function to sort the array in ascending order
void sortAscending(int arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = i + 1; j < size; j++) {
            if (arr[i] > arr[j]) {
                // Swap the elements
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
}
// Function to swap the values at two specified indices
void swapElements(int arr[], int index1, int index2) {
    int temp = arr[index1];
    arr[index1] = arr[index2];
    arr[index2] = temp;
}
```

```
int main() {  
    const int SIZE = 5; // Initial size of the array  
    int numbers[SIZE] = {10, 20, 5, 40, 30}; // Example array  
    // Call the function to swap values 40 and 30  
    int index1 = -1;  
    int index2 = -1;  
    // Find the indices of the values 40 and 30  
    for (int i = 0; i < SIZE; i++) {  
        if (numbers[i] == 40) {  
            index1 = i;  
        }  
        if (numbers[i] == 30) {  
            index2 = i;  
        }  
    }  
    // Swap the elements if both values are found  
    if (index1 != -1 && index2 != -1) {  
        swapElements(numbers, index1, index2);  
    }  
}
```

```
// code continues from previous slide
// Print the array after swapping
cout << "Array-after-swapping-40-and-30:-";
for (int i = 0; i < SIZE; i++) {
    cout << numbers[i] << "-";
}
cout << endl;
// Call the function to sort the array in ascending order
sortAscending(numbers, SIZE);
// Print the sorted array
cout << "Array-sorted-from-lowest-to-highest:-";
for (int i = 0; i < SIZE; i++) {
    cout << numbers[i] << "-";
}
cout << endl;
return 0;
}
```

Array deletion

```
#include <iostream>
using namespace std;

int main() {
    const int SIZE = 5; // Initial size of the array
    int numbers[SIZE] = {10, 20, 5, 40, 30}; // Example array
    int valueToRemove = 5;
    int newSize = SIZE; // Start with the initial size
    int i = 0; // Index for traversing the array
    while (i < newSize) {
        if (numbers[i] == valueToRemove) {
            // Shift elements to the left
            for (int j = i; j < newSize - 1; j++) {
                numbers[j] = numbers[j + 1];
            }
            newSize--; // Reduce the size of the array
        } else {
            i++; // Only increment i if no shift happened
        }
    }
}
```

```
//code continues from previous slide  
// Print the updated array
```

```
    cout << "Array-after-deleting-" << valueToRemove << ":-";  
    for (int k = 0; k < newSize; k++) {  
        cout << numbers[k] << "-";  
    }  
    cout << endl;  
  
    return 0;  
}
```